SaaS Churn Analysis & SQL queries

1. CUSTOMER CHURN & RETENTION DASHBOARD

- **Total customers (Baseline KPI)**

Total Customers

```
SELECT COUNT(*) AS total_customers

FROM customers;
```

total_customers
5000

- **Churn rate**

```
SELECT

COUNT(*) AS total_customers,

SUM(CASE WHEN status = 'Churned' THEN 1 ELSE 0 END) AS churned_customers,

ROUND(

SUM(CASE WHEN status = 'Churned' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2

) AS churn_rate_percentage

FROM customers;
```

total_customers	churned_customers	churn_rate_percentage
5000	408	8.16

- **Average LTV**

```
SELECT

ROUND(AVG(total_revenue),2) AS avg_ltv

FROM (

SELECT

customer_id,

SUM(COALESCE(amount,0)) AS total_revenue

FROM payments

GROUP BY customer_id

) t;
```

avg_ltv
51.44

- **Churn Rate**

```
SELECT
ROUND(
SUM(CASE WHEN status = 'Churned' THEN 1 ELSE 0 END) * 100.0 /
COUNT(*),2) AS churn_rate_percentage
FROM customers;
```

churn_rate_percentage
8.16

- **Churn rate by subscription plan**

```
SELECT
subscription_plan,
COUNT(*) AS total_users,
SUM(
CASE
WHEN status IS NOT NULL
AND LOWER(TRIM(status)) IN ('churned','churn','inactive','yes','1')
THEN 1
ELSE 0
END
) AS churned_users,
ROUND(
SUM(
CASE
WHEN status IS NOT NULL
AND LOWER(TRIM(status)) IN ('churned','churn','inactive','yes','1')
THEN 1
ELSE 0
END
```

```
) * 100.0 / COUNT(*), 2
) AS churn_rate
FROM customers
GROUP BY subscription_plan;
```

subscription_plan	total_users	churned_users	churn_rate
Basic	2513	231	9.19
Standard	1519	106	6.98
Premium	968	71	7.33

2. CHURN DRIVERS

- **Engagement vs Churn**

```
SELECT
c.status,
AVG(u.logins) AS avg_logins,
AVG(u.watch_time_hours) AS avg_watch_time
FROM customers c
JOIN usage_data u
ON c.customer_id = u.customer_id
GROUP BY c.status;
```

status	avg_logins	avg_watch_time
Active	15.0054	39.4793
Churned	14.9835	39.2355

- **Support tickets vs churn**

```
SELECT
CASE
 WHEN COALESCE(u.support_tickets,0) = 0 THEN 'No Issues'
WHEN COALESCE(u.support_tickets,0) BETWEEN 1 AND 3 THEN 'Few Issues'
ELSE 'High Issues'
END AS issue_level,
COUNT(DISTINCT c.customer_id) AS total_users,
SUM(CASE
WHEN LOWER(TRIM(c.status)) = 'churned' THEN 1
```

```
ELSE 0
END) AS churned_users,
ROUND(
SUM(CASE
WHEN LOWER(TRIM(c.sttus)) = 'churned' THEN 1
ELSE 0
END) * 100.0
/ COUNT(DISTINCT c.customer_id), 2
) AS churn_rate
FROM customers c
LEFT JOIN usage_data u
ON c.customer_id = u.customer_id
GROUP BY issue_level
ORDER BY churn_rate DESC;
```

issue_level	total_users	churned_users	churn_rate
Few Issues	5000	3017	60.34
No Issues	4976	1793	36.03
High Issues	966	86	8.9

- **Feedback (NPS) vs churn**

```
SELECT
CASE
WHEN COALESCE(u.support_tickets,0) = 0 THEN 'No Issues'
WHEN COALESCE(u.support_tickets,0) BETWEEN 1 AND 3 THEN 'Few Issues'
ELSE 'High Issues'
END AS issue_level,
COUNT(DISTINCT c.customer_id) AS total_users,
SUM(CASE
WHEN LOWER(TRIM(c.status)) = 'churned' THEN 1
ELSE 0
END) AS churned_users,
ROUND(
```

```
SUM(CASE
WHEN LOWER(TRIM(c.status)) = 'churned' THEN 1
ELSE 0
END) * 100.0
/ COUNT(DISTINCT c.customer_id), 2
) AS churn_rate
FROM customers c
LEFT JOIN usage_data u
ON c.customer_id = u.customer_id
GROUP BY issue_level
ORDER BY churn_rate DESC;
```

issue_level	total_users	churned_users	churn_rate
Few Issues	5000	3017	60.34
No Issues	4976	1793	36.03
High Issues	966	86	8.9

3. CUSTOMER BEHAVIOR

- **Aquisition channel vs churn**

```
SELECT
  acquisition_channel,
  status,
  COUNT(*) AS total_customers
FROM customers
GROUP BY acquisition_channel, status
ORDER BY acquisition_channel;
```

acquisition_channel	status	total_customers
Facebook	Active	706
Facebook	Churned	73
Google Ads	Active	800
Google Ads	Churned	60
Instagram	Active	791
Instagram	Churned	60
Referral	Active	748
Referral	Churned	71
TikTok	Active	789

TikTok	Churned	72
YouTube	Active	758
YouTube	Churned	72

- **Billing cycle vs churn**

```
SELECT
  billing_cycle,
  status,
  COUNT(*) AS total_customers
FROM customers
GROUP BY billing_cycle, status
ORDER BY billing_cycle;
```

billing_cycle	status	total_customers
Annual	Active	1458
Annual	Churned	61
Monthly	Active	3134
Monthly	Churned	347

4. REVENUE AND LTV

- **Revenue by status**

```
SELECT
c.status,
c.subscription_plan,
SUM(p.amount) AS revenue
FROM customers c
JOIN payments p
ON c.customer_id = p.customer_id
GROUP BY c.status, c.subscription_plan;
```

status	subscription_plan	revenue
Active	Basic	104335.56
Active	Standard	78940.23
Churned	Basic	6593.4
Churned	Standard	3520.29
Active	Premium	61593.91
Churned	Premium	2218.52

- **LTV Distribution**

```
SELECT
CASE
WHEN SUM(p.amount) <= 500 THEN '0-500'
WHEN SUM(p.amount) <= 1000 THEN '501-1000'
WHEN SUM(p.amount) <= 1500 THEN '1001-1500'
WHEN SUM(p.amount) <= 2000 THEN '1501-2000'
ELSE '2000+'
END AS ltv_bin,
c.status,
COUNT(DISTINCT c.customer_id) AS total_customers
FROM customers c
LEFT JOIN payments p
ON c.customer_id = p.customer_id
GROUP BY c.customer_id, c.status,
```

- **Monthly Recurring Revenue (MRR)**

```
SELECT
SUM(monthly_fee) AS MRR
FROM customers
WHERE status = 'Active';
```

MRR
54598.08

- **Annual Recurring Revenue (ARR)**

```
SELECT
SUM(monthly_fee) * 12 AS ARR
FROM customers
WHERE status = 'Active';
```

ARR
655177

- **Revenue Lost Due to Churn**

```
SELECT
SUM(monthly_fee) AS revenue_lost
FROM customers
WHERE status = 'Churned';
```

revenue_lost
4748.92

- **Customer life time value (LTV)**

```
SELECT
c.customer_id,
COALESCE(SUM(p.amount), 0) AS total_ltv
FROM customers c
LEFT JOIN payments p
ON c.customer_id = p.customer_id
GROUP BY c.customer_id;
```

- **Average LTV**

```
SELECT
ROUND(AVG(total_revenue),2) AS avg_ltv
FROM (
SELECT
customer_id,
SUM(COALESCE(amount,0)) AS total_revenue
FROM payments
GROUP BY customer_id
) t;
```

- **LTV vs Churn**

```
SELECT
c.status,
ROUND(AVG(p.amount),2) AS avg_revenue
FROM customers c
LEFT JOIN payments p
```

```
ON c.customer_id = p.customer_id
GROUP BY c.status;
```

- **CREATE COHORT (Signup Month)**

```
SELECT
customer_id,
DATE_FORMAT(signup_date, '%Y-%m') AS cohort_month
FROM customers;
```

- **Activity month (from Usage data)**

```
SELECT
customer_id,
month AS activity_month
FROM usage_data;
```

- **Calculate month difference**

```
SELECT
c.customer_id,
DATE_FORMAT(c.signup_date, '%Y-%m') AS cohort_month,
u.month AS activity_month,
TIMESTAMPDIFF(
MONTH,
c.signup_date,
STR_TO_DATE(CONCAT(u.month, '-01'), '%Y-%m-%d')
) AS month_number
FROM customers c
JOIN usage_data u
ON c.customer_id = u.customer_id;
```

- **Cohort size**

```
SELECT
DATE_FORMAT(signup_date, '%Y-%m') AS cohort_month,
COUNT(DISTINCT customer_id) AS cohort_size
FROM customers
GROUP BY cohort_month;
```

```
-- RETENTION TABLE (MAIN QUERY)
SELECT
cohort_month,
month_number,
COUNT(DISTINCT customer_id) AS active_users
FROM (
SELECT
 c.customer_id,
DATE_FORMAT(c.signup_date, '%Y-%m') AS cohort_month,
TIMESTAMPDIFF(
MONTH,
c.signup_date,
STR_TO_DATE(CONCAT(u.month, '-01'), '%Y-%m-%d')
) AS month_number
FROM customers c
JOIN usage_data u
ON c.customer_id = u.customer_id
) tGROUP BY cohort_month, month_number
ORDER BY cohort_month, month_number;
```

- **Retention %**

```
SELECT
DATE_FORMAT(c.signup_date, '%Y-%m') AS cohort_month,
TIMESTAMPDIFF(
MONTH,
c.signup_date,
STR_TO_DATE(CONCAT(u.month, '-01'), '%Y-%m-%d')
) AS month_number,
COUNT(DISTINCT c.customer_id) AS active_users,
cs.total_customers,
ROUND(
COUNT(DISTINCT c.customer_id) / cs.total_customers * 100,
```

```
2
) AS retention_percentage
FROM customers c
JOIN usage_data u
ON c.customer_id = u.customer_id
JOIN (
SELECT
DATE_FORMAT(signup_date, '%Y-%m') AS cohort_month,
COUNT(*) AS total_customers
FROM customers
GROUP BY DATE_FORMAT(signup_date, '%Y-%m')
) cs
ON DATE_FORMAT(c.signup_date, '%Y-%m') = cs.cohort_month
GROUP BY
DATE_FORMAT(c.signup_date, '%Y-%m'),
month_number,
cs.total_customers
ORDER BY
cohort_month,
month_number;
```